

## 5교시: 개발 환경용 Compose 구성

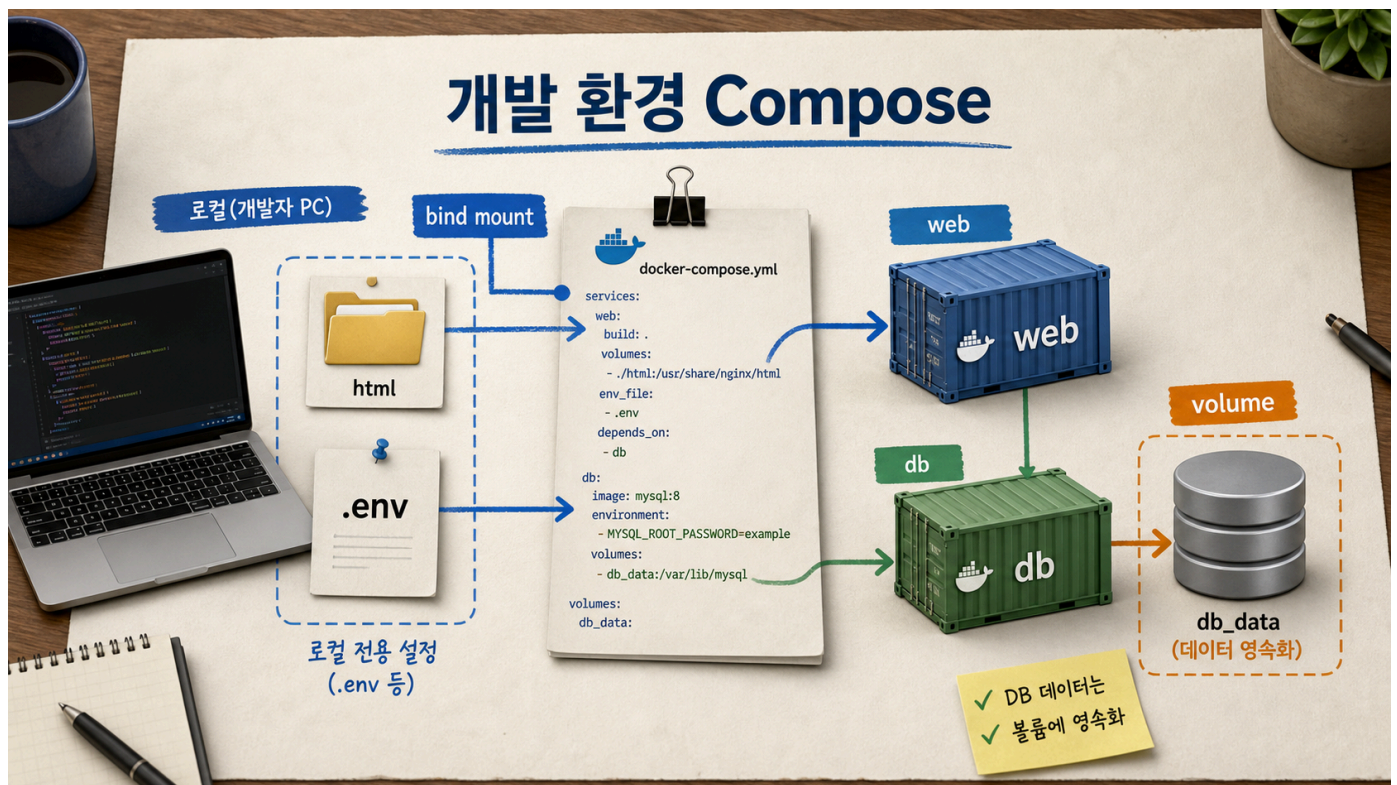
### 수업 목표

- bind mount, `.env`, named volume을 개발 환경 기준으로 구분한다.
- local-only 설정과 production 설정을 섞지 않는 이유를 설명한다.
- README에 개발 실행 조건과 secret 비노출 기준을 기록한다.

### 50분 흐름

| 시간     | 활동                                       | 비중     | 산출                  |
|--------|------------------------------------------|--------|---------------------|
| 0-10분  | 개발 환경에서 바꾸고 싶은 것 정리                      | 설명 20% | change list         |
| 10-22분 | bind mount와 <code>.env</code> 읽기         | 설명 25% | dev config map      |
| 22-35분 | HTML 수정 후 reload 확인                      | 실행 30% | bind mount evidence |
| 35-44분 | <code>.env.example</code> 와 secret 기준 정리 | 설명 15% | security note       |
| 44-50분 | README dev section 작성                    | 실행 10% | handoff draft       |

### Visual 1: 개발용 Compose 구성



이 visual은 host file과 container service가 bind mount와 environment file로 연결되는 흐름을 보여준다.

### 핵심 설명

개발 환경에서는 source file을 바꾸고 바로 확인해야 한다. Day 4 실습의 web service는 `./html:/usr/share/nginx/html:ro` bind mount를 사용한다. image를 다시 build하지 않아도 host의 HTML 파일 변경을 container가 읽을 수 있다.

환경별 값은 `.env`에 둔다. 공개 repository에는 `.env.example`처럼 필요한 key 이름과 예시만 둔다. 실제 password, token, 개인 계정 값은 commit하지 않는다. 실습용 password도 공개 저장소에 반복해서 올리는 습관을 만들면 안 된다.

named volume은 DB data 보존을 위해 사용한다. 개발 중에는 `down -v` 로 초기화가 필요할 수 있지만, 이 명령은 data 삭제다. README에는 "재시작 cleanup"과 "초기화 cleanup"을 구분해서 적는다.

## 실행 명령

```
cd week2/day4/labs/compose-app
docker compose up -d
curl -s http://localhost:18084 | grep compose-site-v1
```

html/index.html 의 정상 확인 문구를 바꾼 뒤 다시 확인한다.

```
curl -s http://localhost:18084 | grep compose-site
```

## 개발 설정 판단표

| 설정                 | 개발에서 쓰는 이유      | 주의                  |
|--------------------|-----------------|---------------------|
| bind mount         | 파일 수정 후 빠른 확인   | host path 의존        |
| .env               | 로컬별 값 분리        | 실제 secret commit 금지 |
| named volume       | DB data 유지      | down -v는 삭제         |
| host port override | port 충돌 회피      | README에 실제 port 기록  |
| profile service    | 도구 container 분리 | 기본 실행 대상인지 구분       |

## 기록 템플릿

```
## Lesson 5 Dev Compose Evidence
- bind mount source:
- bind mount destination:
- env file:
- public example file:
- named volume:
- local-only setting:
- cleanup warning:
```

## 평가 기준

| 기준    | 2점 evidence                |
|-------|----------------------------|
| 개발 설정 | bind mount와 .env의 목적을 설명했다 |
| 보안    | 실제 secret을 공개 파일에 쓰지 않았다   |
| data  | volume cleanup 위험을 구분했다    |

### 공식 근거 링크

- Compose services reference: <https://docs.docker.com/reference/compose-file/services/>
- Twelve-Factor App Config: <https://12factor.net/config>

## 선행 지식과 범위 경계

이 교시는 "개발 환경을 편하게 만드는 설정"과 "운영 환경에 가져가면 위험한 설정"을 구분하는 시간이다. bind mount와 .env 는 개발 생산성을 높이지만, 그대로 production 기준이 되는 것은 아니다.

학생은 Day 3에서 bind mount와 named volume의 차이를 배웠다. Day 4에서는 그 차이를 Compose file 안에서 읽고, README에 local-only 설정으로 기록하는 법을 배운다.

## 학술 기준 연결

이 교시는 전문적 책임과 직무 태도(disposition)를 다룬다. 기술적으로는 bind mount와 env file을 사용하는 것이지만, 교육적으로는 secret 비노출, 재현성, 환경 분리 책임을 배우는 시간이다.

| 기준                               | 적용                                 |
|----------------------------------|------------------------------------|
| ABET professional responsibility | secret과 data 삭제 위험을 설명             |
| CS2023 Skill                     | 개발 환경 실행 조건을 Compose로 구성           |
| CS2023 Disposition               | 편의보다 안전한 handoff를 우선               |
| NIST NICE                        | configuration과 credential 노출 위험 인식 |

## 개발 환경과 운영 환경의 차이

개발 환경은 빠른 변경과 관찰이 중요하다. 운영 환경은 통제된 artifact, 최소 권한, 감사 가능성이 중요하다.

| 항목        | 개발 환경              | 운영 환경                           |
|-----------|--------------------|---------------------------------|
| source 반영 | bind mount로 빠르게 확인 | image build 후 배포                |
| env 값     | .env 로컬 주입         | secret manager 또는 안전한 배포 시스템    |
| DB data   | 초기화와 재생성 가능        | backup, retention, migration 필요 |
| port      | 충돌 피해서 자유롭게 조정     | ingress/load balancer 정책        |
| logging   | local logs 중심      | centralized logging/metrics     |

Day 4는 개발 환경을 다루지만, 운영 환경과 혼동하지 않도록 "local-only"라고 명시한다.

## bind mount 판단 기준

bind mount는 host path와 container path를 직접 연결한다. 개발 중 HTML, config sample, source file을 빠르게 바꿔 볼 때 유용하다.

하지만 bind mount는 host directory 구조에 의존한다. 다른 사람이 repository를 다른 경로에 clone해도 상대 경로가 맞으면 동작하지만, 절대 경로를 쓰면 handoff가 약해진다.

좋은 예:

```
volumes:
  - ./html:/usr/share/nginx/html:ro
```

나쁜 예:

```
volumes:
  - /Users/someone/Desktop/my-html:/usr/share/nginx/html
```

절대 경로는 개인 환경이 드러나고 재현성이 낮다.

## .env.example 의 역할

`.env.example` 은 실제 secret을 담은 파일이 아니라 "어떤 설정 이름이 필요한가"를 알려주는 계약이다.

좋은 `.env.example` :

```
WEB_PORT=18084
POSTGRES_DB=paperclip
POSTGRES_USER=paperclip
POSTGRES_PASSWORD=change-me-locally
```

실제 수업에서는 `change-me-locally` 를 로컬 실습용 값으로 바꾼다. 공개 repository에 개인 password, Docker Hub token, cloud access key를 쓰지 않는다.

## named volume 판단 기준

DB는 container가 삭제되어도 data가 남아야 하는 경우가 많다. 그래서 named volume을 사용한다.

| 상황                        | 권장                                          |
|---------------------------|---------------------------------------------|
| 정적 HTML 파일 제공             | bind mount 또는 image COPY                    |
| 개발 중 source 변경 확인         | bind mount                                  |
| PostgreSQL data directory | named volume                                |
| 일회성 cache                 | 익명 volume 또는 삭제 가능 storage                  |
| production database       | Compose volume만으로 충분하지 않고 backup/restore 필요 |

## 실무 risk classification

| 위험                           | 가능성 | 영향 | 완화                                                     |
|------------------------------|-----|----|--------------------------------------------------------|
| <code>.env</code> commit     | 중간  | 높음 | <code>.gitignore</code> , <code>.env.example</code> 사용 |
| <code>down -v</code> data 삭제 | 중간  | 높음 | cleanup 문서 분리                                          |
| bind mount 경로 오류             | 높음  | 중간 | 상대 경로와 config 검증                                       |
| host port 충돌                 | 높음  | 낮음 | <code>WEB_PORT</code> 변수화                              |
| 운영 설정과 개발 설정 혼동              | 중간  | 높음 | local-only 주석과 README 분리                               |

## 실습 확장: port override

`.env` 의 `WEB_PORT` 를 바꿔 실행한다.

```
WEB_PORT=18085 docker compose --env-file .env.example config
```

확인할 점:

- published port가 18085 로 바뀌는가?
- container port 80 은 그대로인가?
- README에는 실제 사용한 host port를 기록했는가?

## 오해 점검 문항

| 문항                                               | 기대 답                       |
|--------------------------------------------------|----------------------------|
| <code>.env.example</code> 에 실제 password를 넣어도 되는가 | 아니다. 예시 또는 placeholder만 둔다 |

|                                      |                                        |
|--------------------------------------|----------------------------------------|
| bind mount는 image rebuild를 대체하는가     | 개발 중 빠른 확인에는 유용하지만 배포 artifact와 다르다    |
| named volume은 container와 함께 자동 삭제되는가 | 아니다. down -v 등 별도 삭제가 필요하다             |
| read-only bind mount는 왜 쓰는가          | container가 host source를 수정하지 못하게 하기 위해 |

Evidence 수준 구분

| 수준          | 예시                                                                  |
|-------------|---------------------------------------------------------------------|
| 약한 evidence | .env 사용한다고 적음                                                       |
| 중간 evidence | .env.example이 있음                                                    |
| 강한 evidence | required variable, secret 주의, local-only 설정, cleanup 위험을 README에 기록 |

전이 과제

자기 프로젝트에 local-only Compose section을 추가한다고 가정하고 다음을 쓴다.

```
## Local Development Only
- Uses bind mount for:
- Uses `.env` for:
- Does not commit:
- Data volume:
- Safe cleanup:
- Destructive cleanup:
```

이 과제는 실무 README에서 개발 환경과 운영 환경을 섞지 않는 연습이다.

종료 전 확인 문장

bind mount와 `.env`는 개발 속도를 높이지만, 공개 repository와 운영 환경에서는 노출, 경로 의존성, data 삭제 위험을 별도로 통제해야 한다.

이 문장을 README 주의사항으로 바꿔 쓸 수 있으면 Lesson 5의 실무 목표를 달성한 것이다.